

Speech Signal Processing BSAI – 7th Semester

Lab 4 Task Report

Submitted by: Junaid Asif

Roll No: BSAI-144

Instructor: Dr. Sajjad Ghauri

October 09, 2025

Objective

To study and implement convolution, Discrete Fourier Transform (DFT), Inverse Discrete Fourier Transform (IDFT), and Fast Fourier Transform (FFT) in MATLAB, and to analyze their effects on different discrete-time signals.

Task 1: Convolution without Using Built-in Function

MATLAB Code:

```
1 clc;
2 close all;
3 clear all;
4
5 x = input('Enter x: ');
6 h = input('Enter h: ');
7
8 m = length(x);
9 n = length(h);
10
11 X = [x, zeros(1, n)];
12 H = [h, zeros(1, m)];
13
14 for i = 1:m+n-1
15 Y(i) = 0;
16 for j = 1:m
17 if (i-j+1 > 0)
18 Y(i) = Y(i) + X(j) * H(i-j+1);
19 end
20 end
21 end
22
23 Y
24 stem(Y);
25 ylabel('Y[n]');
26 xlabel('n');
27 title('Convolution of Two Signals without conv function');
```

Explanation: code manually computes the convolution of two discrete-time sequences by summing overlapping samples. The output length is $m + n - 1$. The stem plot illustrates the discrete convolution result.

Task 2: Discrete Fourier Transform (DFT)

MATLAB Code:

```
1 clc;
2 clear all;
3 close all;
4
5 a = input('Enter the sequence :');
6 N = length(a);
7 disp('The length of the sequence is:'); N
8
9 for k = 1:N
```

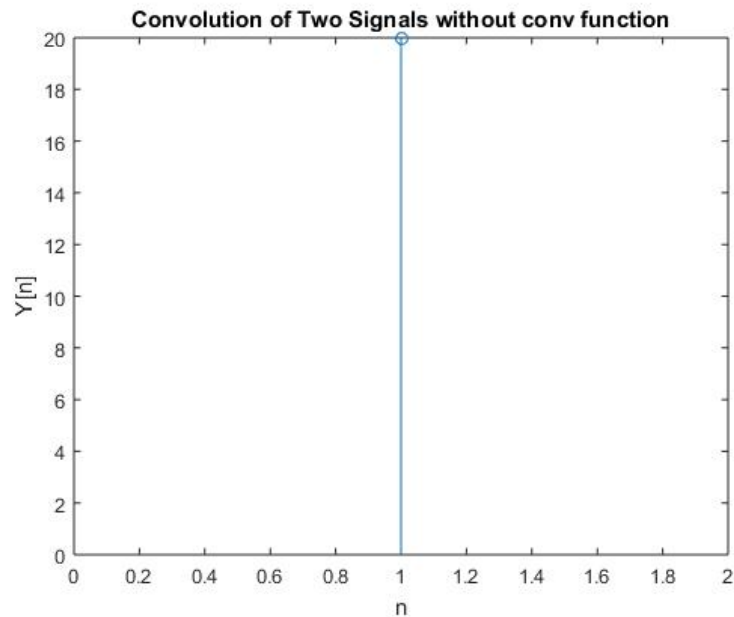


Figure 1: Convolution of Two Signals without `conv()` Function

```

10 y(k) = 0;
11 for i = 1:N
12 y(k) = y(k) + a(i) * exp((-2*pi*j/N) * ((i-1)*(k-1)));
13 end
14 end
15
16 k = 1:N;
17 disp('The result is:'); y
18
19 figure(1);
20 subplot(211);
21 stem(k, abs(y));
22 xlabel('Sample values n');
23 ylabel('Amplitude');
24 title('Magnitude response of the DFT');
25
26 subplot(212);
27 stem(k, angle(y)*180/pi);
28 xlabel('Sample values n');
29 ylabel('Phase');
30 title('Phase response of the DFT');

```

Explanation: DFT converts a discrete-time signal from the time domain into its frequency-domain representation. The magnitude spectrum shows frequency amplitudes, while the phase spectrum represents phase shifts.

Task 3: Inverse Discrete Fourier Transform (IDFT)

MATLAB Code:

```

1 clc;
2 clear all;
3 close all;

```

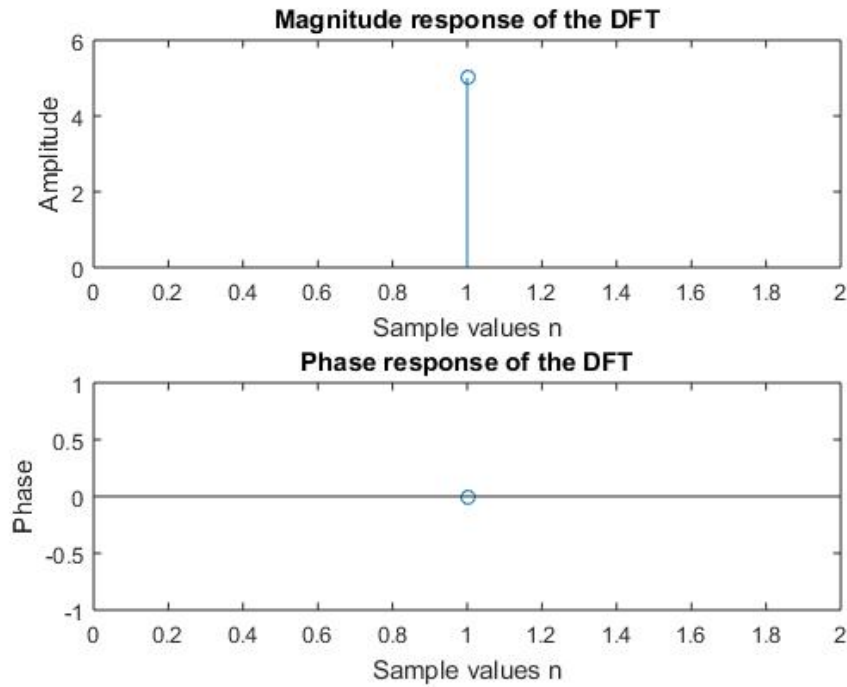


Figure 2: Magnitude and Phase Response of DFT

```

4
5 a = input('Enter the sequence :');
6 N = length(a);
7 disp('The length of the sequence is:'); N
8
9 for n = 1:N
10 y(n) = 0;
11 for k = 1:N
12 y(n) = y(n) + a(k) * exp((2*pi*1i*(k-1)*(n-1))/N);
13 end
14 end
15
16 n = 1:N;
17 y1 = (1/N) * y;
18 disp('The result is:'); y1
19
20 figure(1);
21 stem(n, y1);
22 xlabel('Sample values n');
23 ylabel('Amplitude');
24 title('Magnitude response of the IDFT');

```

Explanation: IDFT reconstructs the original discrete-time signal from its frequency-domain representation, verifying that DFT and IDFT are inverse operations.

Task 4: Fast Fourier Transform (FFT)

MATLAB Code:

```

1 clc;

```

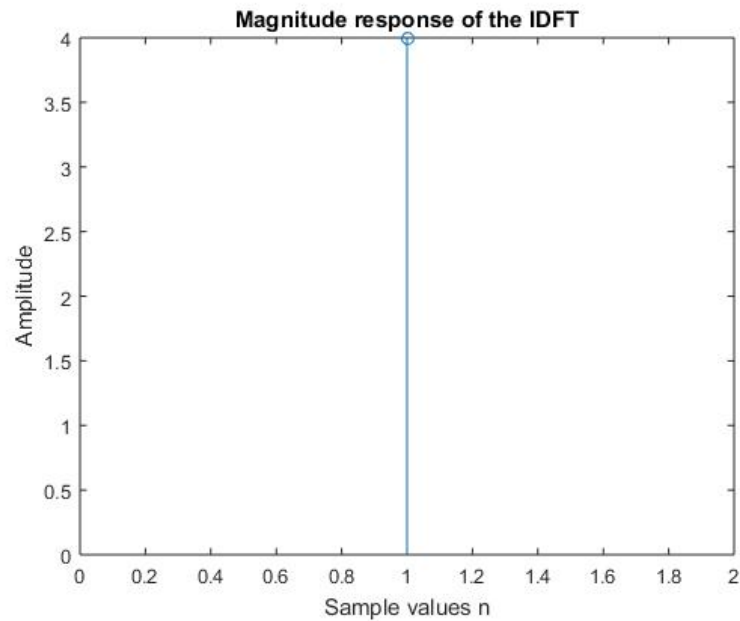


Figure 3: Inverse Discrete Fourier Transform (IDFT) Output

```

2 clear all;
3 close all;
4
5 t = -2:1:2;
6 y = [zeros(1,2), 1, zeros(1,2)];
7
8 subplot(3,1,1);
9 stem(t, y);
10 grid;
11 title('Impulse Response');
12 xlabel('Time');
13 ylabel('Amplitude');
14
15 N = input('Enter the length of the FFT sequence: ');
16 xk = fft(y, N);
17 magxk = abs(xk);
18 angxk = angle(xk);
19 k = 0:N-1;
20
21 subplot(3,1,2);
22 stem(k, magxk);
23 xlabel('k');
24 ylabel('|x(k)|');
25
26 subplot(3,1,3);
27 stem(k, angxk);
28 xlabel('k');
29 ylabel('arg(x(k))');

```

Explanation: efficiently computes the DFT using a fast algorithm. The plots show the impulse in time domain and its corresponding magnitude and phase spectra.

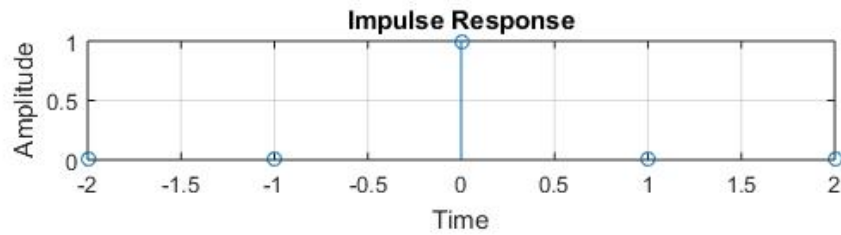


Figure 4: Fast Fourier Transform (FFT) Output

Task 5: DFT of Two Sequences

MATLAB Code:

```

1 clc;
2 clear;
3 close all;
4
5 x = [1, 2, 3, 4];
6 h = [4, 3, 2, 1];
7
8 N = length(x);
9
10 X = zeros(1, N);
11 for k = 1:N
12 for n = 1:N
13 X(k) = X(k) + x(n)*exp(-1i*2*pi*(k-1)*(n-1)/N);
14 end
15 end
16
17 H = zeros(1, N);
18 for k = 1:N
19 for n = 1:N
20 H(k) = H(k) + h(n)*exp(-1i*2*pi*(k-1)*(n-1)/N);
21 end
22 end
23
24 k = 0:N-1;
25
26 figure;
27 subplot(2,2,1);
28 stem(k, abs(X),'filled');
```

```

29 title('Magnitude of DFT of x');
30
31 subplot(2,2,2);
32 stem(k, angle(X)*180/pi,'filled');
33 title('Phase of DFT of x');
34
35 subplot(2,2,3);
36 stem(k, abs(H),'filled');
37 title('Magnitude of DFT of h');
38
39 subplot(2,2,4);
40 stem(k, angle(H)*180/pi,'filled');
41 title('Phase of DFT of h');

```

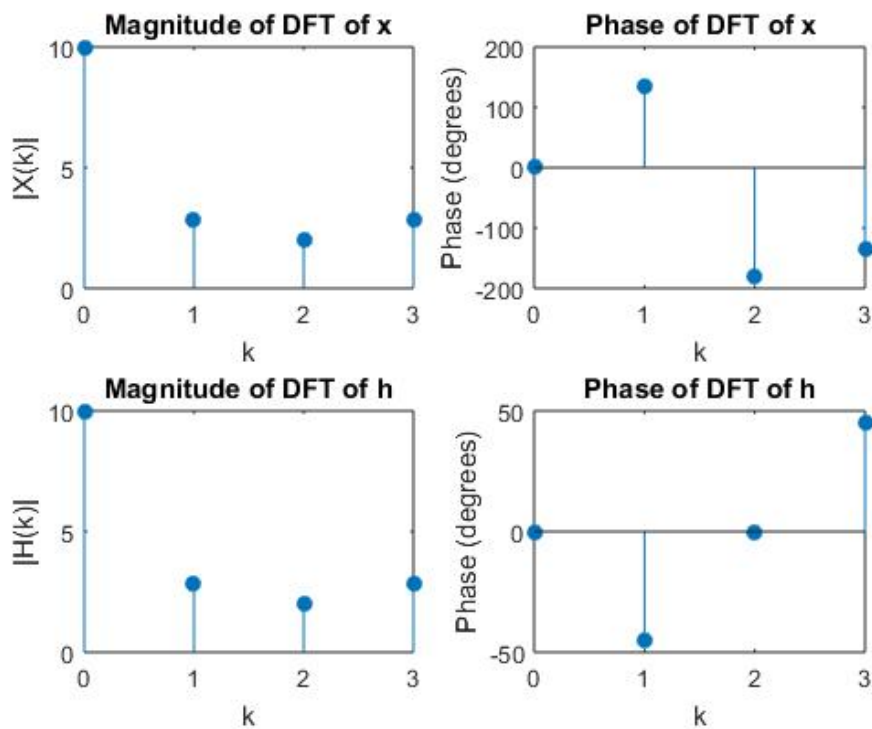


Figure 5: DFT Magnitude and Phase Plots for Two Sequences

Explanation: task compares the frequency-domain magnitude and phase characteristics of two discrete-time sequences.

Task 6: FFT of Step, Ramp, and Exponential Sequences

MATLAB Code:

```

1 clc;
2 clear;
3 close all;
4
5 N = 16;

```

```

6 n = 0:N-1;
7
8 u = ones(1, N);
9 r = n;
10 a = 0.8;
11 e = a.^n;
12
13 U = fft(u);
14 R = fft(r);
15 E = fft(e);

```

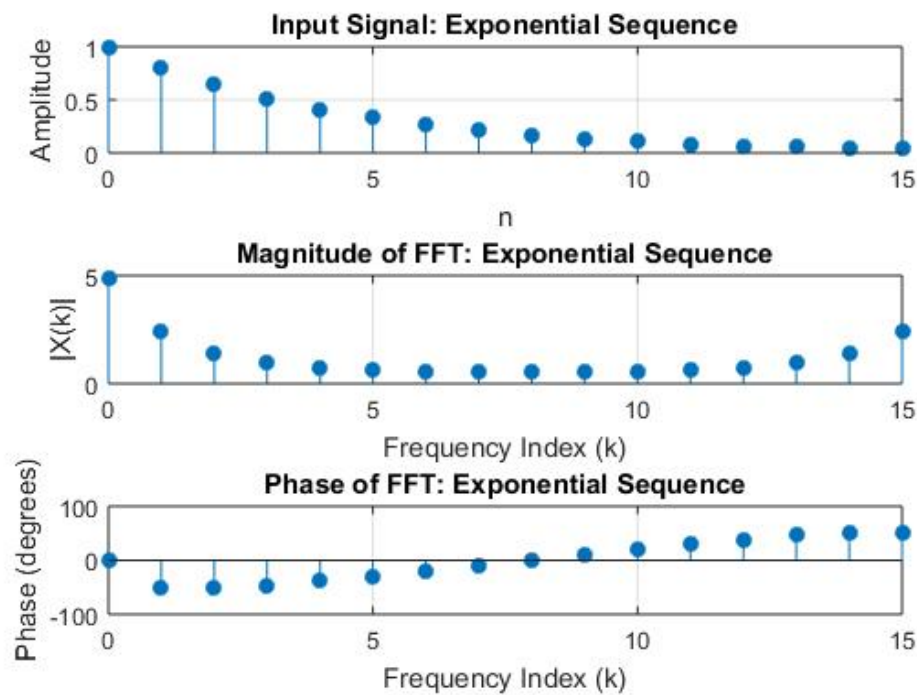


Figure 6: FFT of Step, Ramp, and Exponential Sequences

Explanation:

- **Step sequence:** Dominant low-frequency components.
- **Ramp sequence:** Linear variation reflected in phase.
- **Exponential sequence:** Decaying magnitude spectrum.

Conclusion

In this lab, convolution, DFT, IDFT, and FFT were successfully implemented and analyzed using MATLAB. Convolution demonstrated time-domain signal interaction, while DFT and IDFT illustrated frequency-domain representation and reconstruction. FFT highlighted computational efficiency for spectral analysis. These experiments strengthened understanding of discrete-time signal processing concepts.